AetherCore Agent Operability Report

Executive Summary

AetherCore is designed as a sophisticated AI agent for code auditing and application building. While the core architecture and 3D visual layer are impressive, several critical functional and security gaps must be addressed to transform it into a fully operative production-ready agent. This report outlines the necessary fixes and recommended additions.

1. Critical Technical Fixes

Issue Category	Description	Recommended Action
Security	Hardcoded NVIDIA NIM API key in `server.ts` (Line 32).	Remove hardcoded keys immediately. Implement a secure `.env` management system and validate keys at startup.
Stability	Model name `gemini-2.5-pro` is invalid for the current SDK.	Update model reference to `gemini-1.5-pro` or `gemini-2.0-flash` to ensure fallback functionality works.
Error Handling	Fragile mode switching logic based on string matching in the summary.	Implement a dedicated `type` field in the API response (e.g., `mode: “build”

2. Functional Enhancements for Operability

2.1 State Management & Context

The current implementation only supports single-turn interactions. To be fully operative, the agent requires **Conversation Memory**. This involves maintaining a message history on the server or in the Zustand store and passing it to the LLM in subsequent requests. This will allow users to ask follow-up questions or request refinements to the generated code.

2.2 Dynamic Visualizations

The 3D “Blueprint” component is currently a static representation. To provide real value, the agent should generate a **Structural Schema** as part of its JSON response. The frontend should

then parse this schema to render a dynamic 3D map of the actual code architecture, highlighting modified components or new modules.

2.3 File System Integration

A professional developer agent must be able to interact with the project's environment. We recommend adding **Server-Side File Operations** that allow the agent to:

- Read multiple files from the project directory to provide better context.
- Write the "solutionCode" directly to files upon user confirmation.
- Run basic linting or test commands and report the results back to the UI.

3. UI/UX Improvements

The user interface, while visually stunning, lacks several professional features. Replacing `react-simple-code-editor` with a more robust solution like **Monaco Editor** would provide syntax highlighting, auto-completion, and better handling of large files. Additionally, the PDF export feature should be expanded to include a timestamped audit trail and a more detailed breakdown of performance metrics.

4. Proposed Roadmap

1. **Phase 1: Stabilization (Week 1)** - Fix API key handling, update model versions, and implement robust error boundaries.
2. **Phase 2: Intelligence (Week 2)** - Implement multi-turn conversation history and context-aware file reading.
3. **Phase 3: Visualization (Week 3)** - Connect the 3D canvas to actual code analysis data for dynamic architectural mapping.
4. **Phase 4: Deployment (Week 4)** - Finalize production build scripts and implement secure user authentication for API key management.